

Cognizant Digital Nurture hands on--Angular

Complete Code Implementation & Step-by-Step Task Solutions

Project Context: All task solutions build upon a unified application: **Student Course Portal**. File structures reflect standard Angular 20 standalone & module architectures.

HANDS-ON 1: Environment Setup, Project Structure & First Component

Task 1: Scaffold and Explore the Angular Project

Steps 1-3: Scaffold Application & Configuration Explanations

Command used for scaffolding:

```
ng new student-course-portal --routing --style=css
```

Content of `notes.txt` explaining the 8 core setup files:

`notes.txt`

```
angular.json - Main CLI configuration file for build, serve, test targets, asset paths, and
environment settings.
tsconfig.json - Root TypeScript compiler settings and path aliases shared across the workspace.
tsconfig.app.json - Application-specific TypeScript config extending root settings for app
compilation.
package.json - Tracks project dependencies, scripts (start, build, test), and package metadata.
src/main.ts - Application entry point that bootstraps either the AppModule or root Component.
src/app/app.module.ts (or app.config.ts) - Defines root module dependencies or root standalone app
configuration.
src/app/app.component.ts - The root component logic that serves as the main shell of the portal.
src/index.html - Single main HTML page hosting the app-root component and loading global CSS/scripts.
```

Step 4 & 5: Build Output & Production Budget Analysis

Build Command: `ng build` produces the production bundles inside `dist/student-course-portal/browser/`.

Compiled Application Code File: Main compiled application logic resides in `main-[hash].js`. Framework runtime logic resides in `runtime-[hash].js`, polyfills in `polyfills-[hash].js`, and styles in `styles-[hash].css`.

Budget Threshold Explanation: Located in `angular.json` under `architect > build > configurations > production > budgets`:

- `maximumWarning` (e.g., 500kB): Emits a build warning if the bundle size exceeds this threshold.
- `maximumError` (e.g., 1MB): Fails the build execution with an error if the bundle size exceeds this limit.

Task 2: Create and Organise Components

Step 6: Generate Core Components

```
ng generate component components/header
ng generate component pages/home
ng generate component pages/course-list
ng generate component pages/student-profile
```

Step 7: HeaderComponent Implementation

`src/app/components/header/header.component.html`

```
<nav class="header-nav">
  <div class="brand">Student Course Portal</div>
  <ul class="nav-links">
    <li><a routerLink="/">Home</a></li>
    <li><a routerLink="/courses">Courses</a></li>
    <li><a routerLink="/profile">Profile</a></li>
  </ul>
</nav>
```

Step 8: HomeComponent Implementation

src/app/pages/home/home.component.html

```
<div class="home-container">
  <h1>Welcome to Student Course Portal</h1>
  <p>Manage your academic journey, explore available courses, track your GPA, and request
  enrollments.</p>

  <div class="stats-row">
    <div class="stat-card"><h3>Courses Available</h3><p>12</p></div>
    <div class="stat-card"><h3>Enrolled</h3><p>3</p></div>
    <div class="stat-card"><h3>GPA</h3><p>3.8</p></div>
  </div>
</div>
```

Step 9 & 10: Root App Shell Template

src/app/app.component.html

```
<app-header></app-header>
<main class="container">
  <router-outlet></router-outlet>
</main>
```

HANDS-ON 2: Data Binding, Lifecycle Hooks & Component Communication

Task 1: All Four Binding Types

Steps 11-15: Binding Types & Difference Explanation

src/app/pages/home/home.component.ts

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-home',
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.css']
})
export class HomeComponent {
  portalName: string = 'Student Course Portal';
  isPortalActive: boolean = true;
  message: string = '';
  searchTerm: string = '';

  onEnrollClick(): void {
    this.message = 'Enrollment opened!';
  }
}
/*
  DIFFERENCE COMMENT (Step 15):
  [property] is one-way binding flowing data from component class property to DOM element property.
  [(ngModel)] is two-way binding that syncs DOM changes back to component property while also updating
  DOM on component state changes.
*/
```

src/app/pages/home/home.component.html

```
<h1>{{ portalName }}</h1>

<button [disabled]="!isPortalActive" (click)="onEnrollClick()">Enroll Now</button>
<p *ngIf="message">{{ message }}</p>

<div class="search-box">
  <input [(ngModel)]="searchTerm" placeholder="Search..." />
  <p>Searching for: {{ searchTerm }}</p>
</div>
```

Task 2: Lifecycle Hooks

Steps 16-17: HomeComponent Lifecycle Implementation

src/app/pages/home/home.component.ts (Updated)

```
import { Component, OnInit, OnDestroy } from '@angular/core';

@Component({ selector: 'app-home', templateUrl: './home.component.html' })
export class HomeComponent implements OnInit, OnDestroy {
  portalName = 'Student Course Portal';
  isPortalActive = true;
  message = '';
  searchTerm = '';

  ngOnInit(): void {
    console.log('HomeComponent initialised - courses loaded');
  }

  ngOnDestroy(): void {
    console.log('HomeComponent destroyed');
  }

  onEnrollClick(): void { this.message = 'Enrollment opened!'; }
}
```

Step 18-19: CourseCardComponent with ngOnChanges

src/app/components/course-card/course-card.component.ts

```
import { Component, Input, OnChanges, SimpleChanges } from '@angular/core';

@Component({
  selector: 'app-course-card',
  templateUrl: './course-card.component.html'
})
export class CourseCardComponent implements OnChanges {
  @Input() course: any;

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['course']) {
      console.log('Course Input Changed:', {
        previousValue: changes['course'].previousValue,
        currentValue: changes['course'].currentValue
      });
    }
  }
}
```

Task 3: @Input and @Output - Parent-Child Communication

Steps 20-24: Strongly Typed Input, Output Event & Parent Handling

src/app/components/course-card/course-card.component.ts (Full)

```
import { Component, Input, Output, EventEmitter, OnChanges, SimpleChanges } from '@angular/core';

export interface CourseData {
  id: number;
  name: string;
  code: string;
  credits: number;
}

@Component({
  selector: 'app-course-card',
  templateUrl: './course-card.component.html'
})
export class CourseCardComponent implements OnChanges {
  @Input() course!: CourseData;
  @Output() enrollRequested = new EventEmitter<number>();

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['course']) {
      console.log('Course changed:', changes['course'].currentValue);
    }
  }

  onEnroll(): void {
    this.enrollRequested.emit(this.course.id);
  }
}
```

src/app/components/course-card/course-card.component.html

```
<div class="course-card">
  <h3>{{ course.name }} ({{ course.code }})</h3>
  <p>Credits: {{ course.credits }}</p>
  <button (click)="onEnroll()">Enroll</button>
</div>
```

src/app/pages/course-list/course-list.component.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-course-list',
  templateUrl: './course-list.component.html'
})
export class CourseListComponent {
  selectedCourseId: number | null = null;

  courses = [
    { id: 101, name: 'Angular Framework', code: 'CS201', credits: 4 },
    { id: 102, name: 'Data Structures', code: 'CS102', credits: 3 },
    { id: 103, name: 'Database Systems', code: 'CS204', credits: 4 },
    { id: 104, name: 'Web Development', code: 'CS105', credits: 3 },
    { id: 105, name: 'Software Engineering', code: 'CS301', credits: 3 }
  ];

  onEnroll(courseId: number): void {
    console.log('Enrolling in course: ' + courseId);
    this.selectedCourseId = courseId;
  }
}
```

src/app/pages/course-list/course-list.component.html

```
<div class="course-list">
  <app-course-card
    *ngFor="let c of courses"
    [course]="c"
    (enrollRequested)="onEnroll($event)">
  </app-course-card>

  <p *ngIf="selectedCourseId">Selected course ID: {{ selectedCourseId }}</p>
</div>
```

HANDS-ON 3: Directives & Pipes - Built-in and Custom

Task 1: Structural Directives

Steps 25-28: *ngIf, *ngFor, trackBy, *ngSwitch & ng-template

src/app/pages/course-list/course-list.component.ts

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-course-list',
  templateUrl: './course-list.component.html'
})
export class CourseListComponent implements OnInit {
  isLoading: boolean = true;
  selectedCourseId: number | null = null;

  courses: any[] = [];

  ngOnInit(): void {
    setTimeout(() => {
      this.courses = [
        { id: 1, name: 'Angular v20', code: 'CS201', credits: 4, gradeStatus: 'passed' },
        { id: 2, name: 'Node.js Design', code: 'CS202', credits: 3, gradeStatus: 'failed' },
        { id: 3, name: 'Cloud Computing', code: 'CS303', credits: 2, gradeStatus: 'pending' }
      ];
      this.isLoading = false;
    }, 1500);
  }

  // trackBy function avoids DOM re-rendering for unchanged array items
  trackByCourseId(index: number, course: any): number {
    return course.id;
  }
}
```

src/app/pages/course-list/course-list.component.html

```
<div *ngIf="isLoading; else courseGrid">
  <p>Loading courses...</p>
</div>

<ng-template #courseGrid>
  <div *ngIf="courses.length > 0; else noCourses" class="grid">
    <app-course-card
      *ngFor="let course of courses; let i = index; trackBy: trackByCourseId"
      [course]="course">
    </app-course-card>
  </div>
</ng-template>

<ng-template #noCourses>
  <p>No courses available.</p>
</ng-template>
```

Task 2: Attribute Directives ngClass and ngStyle

Steps 29-32: ngClass, Dynamic ngStyle & Getters

src/app/components/course-card/course-card.component.ts

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-course-card',
  templateUrl: './course-card.component.html',
  styleUrls: ['./course-card.component.css']
})
export class CourseCardComponent {
  @Input() course: any;
  isEnrolled: boolean = false;
  isExpanded: boolean = false;

  // Getter keeps template concise and maintains testability in component class
  get cardClasses() {
    return {
      'card--enrolled': this.isEnrolled,
      'card--full': this.course?.credits >= 4,
      'expanded': this.isExpanded
    };
  }

  getBorderColor(): string {
    switch (this.course?.gradeStatus) {
      case 'passed': return 'green';
      case 'failed': return 'red';
      case 'pending': return 'grey';
      default: return 'transparent';
    }
  }

  toggleExpand(): void {
    this.isExpanded = !this.isExpanded;
  }
}
```

src/app/components/course-card/course-card.component.html

```
<div [ngClass]="cardClasses" [ngStyle]="{'border-left': '5px solid ' + getBorderColor()}"
class="card">
  <h3>{{ course.name }}</h3>

  <div [ngSwitch]="course.gradeStatus">
    <span *ngSwitchCase="'passed'" class="badge badge-green">Passed</span>
    <span *ngSwitchCase="'failed'" class="badge badge-red">Failed</span>
    <span *ngSwitchCase="'pending'" class="badge badge-grey">Pending</span>
  </div>

  <button (click)="toggleExpand()">Show Details</button>
</div>
```

Task 3: Custom Directive and Custom Pipe

Steps 33-37: Custom Highlight Directive & CreditLabel Pipe

src/app/directives/highlight.directive.ts

```
import { Directive, ElementRef, HostListener, Input, Renderer2 } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  @Input() appHighlight: string = 'yellow';

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  @HostListener('mouseenter') onMouseEnter(): void {
    this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', this.appHighlight || 'yellow');
  }

  @HostListener('mouseleave') onMouseLeave(): void {
    this.renderer.removeStyle(this.el.nativeElement, 'backgroundColor');
  }
}
```

src/app/pipes/credit-label.pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'creditLabel'
})
export class CreditLabelPipe implements PipeTransform {
  transform(value: number | null | undefined): string {
    if (value === null || value === undefined || value === 0) {
      return 'No Credits';
    }
    if (value === 1) {
      return '1 Credit';
    }
    return `${value} Credits`;
  }
}
```

HANDS-ON 4: Template-Driven Forms & Validation

Task 1: Build the Enrollment Request Form

Task 2: Validation and Error Messages

Steps 38-47: Template-Driven Form with Built-in Validators

src/app/pages/enrollment-form/enrollment-form.component.ts


```

import { Component } from '@angular/core';
import { NgForm } from '@angular/forms';

@Component({
  selector: 'app-enrollment-form',
  templateUrl: './enrollment-form.component.html',
  styleUrls: ['./enrollment-form.component.css']
})
export class EnrollmentFormComponent {
  submitted: boolean = false;

  formData = {
    studentName: '',
    studentEmail: '',
    courseId: null,
    preferredSemester: 'Odd',
    agreeToTerms: false
  };

  onSubmit(form: NgForm): void {
    console.log('Form Value:', form.value);
    console.log('Form Valid:', form.valid);
    if (form.valid) {
      this.submitted = true;
    }
  }
}

```

src/app/pages/enrollment-form/enrollment-form.component.html

```

<form #enrollForm="ngForm" (ngSubmit)="onSubmit(enrollForm)" novalidate>
  <div class="form-group">
    <label>Student Name:</label>
    <input type="text" name="studentName" [(ngModel)]="formData.studentName" required minlength="3"
#nameCtrl="ngModel" />
    <div *ngIf="nameCtrl.touched && nameCtrl.errors" class="error">
      <span *ngIf="nameCtrl.errors['required']">Name is required.</span>
      <span *ngIf="nameCtrl.errors['minlength']">Name must be at least 3 characters.</span>
    </div>
  </div>

  <div class="form-group">
    <label>Student Email:</label>
    <input type="email" name="studentEmail" [(ngModel)]="formData.studentEmail" required email
#emailCtrl="ngModel" />
    <div *ngIf="emailCtrl.touched && emailCtrl.errors" class="error">
      <span *ngIf="emailCtrl.errors['required']">Email is required.</span>
      <span *ngIf="emailCtrl.errors['email']">Invalid email format.</span>
    </div>
  </div>

  <div class="form-group">
    <label>Course ID:</label>
    <input type="number" name="courseId" [(ngModel)]="formData.courseId" required
#courseCtrl="ngModel" />
  </div>

  <div class="form-group">
    <label>Preferred Semester:</label>
    <select name="preferredSemester" [(ngModel)]="formData.preferredSemester">
      <option value="Odd">Odd</option>
      <option value="Even">Even</option>
    </select>
  </div>

  <div class="form-group">
    <label>
      <input type="checkbox" name="agreeToTerms" [(ngModel)]="formData.agreeToTerms" required />
      Agree to Terms
    </label>
  </div>

  <button type="submit" [disabled]="enrollForm.invalid">Submit</button>
  <button type="button" (click)="enrollForm.resetForm()">Reset</button>
</form>

<div *ngIf="submitted" class="success-alert">
  Enrollment request submitted successfully!
</div>

```

[src/app/pages/enrollment-form/enrollment-form.component.css](#)

```

.ng-invalid.ng-touched { border-color: red; }
.ng-valid.ng-touched { border-color: green; }
.error { color: red; font-size: 8pt; }
.success-alert { background: #dcfce7; color: #15803d; padding: 10px; margin-top: 10px; }

```

HANDS-ON 5: Reactive Forms & Custom Validators

Task 1 & Task 2: Reactive Forms, Custom Sync/Async Validators & FormArray

Steps 48-57: Reactive Form Implementation

```
src/app/pages/reactive-enrollment-form/reactive-enrollment-form.component.ts
```

```

import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, FormArray, Validators, AbstractControl, ValidationErrors } from
 '@angular/forms';

// Custom Synchronous Validator
export function noCourseCode(control: AbstractControl): ValidationErrors | null {
  const value = String(control.value || '');
  if (value.startsWith('XX')) {
    return { noCourseCode: true };
  }
  return null;
}

// Custom Asynchronous Validator
export function simulateEmailCheck(control: AbstractControl): Promise<ValidationErrors | null> {
  return new Promise((resolve) => {
    setTimeout(() => {
      if (control.value && control.value.includes('test@')) {
        resolve({ emailTaken: true });
      } else {
        resolve(null);
      }
    }, 800);
  });
}

@Component({
  selector: 'app-reactive-enrollment-form',
  templateUrl: './reactive-enrollment-form.component.html'
})
export class ReactiveEnrollmentFormComponent implements OnInit {
  enrollForm!: FormGroup;

  constructor(private fb: FormBuilder) {}

  ngOnInit(): void {
    this.enrollForm = this.fb.group({
      studentName: ['', [Validators.required, Validators.minLength(3)]],
      studentEmail: ['', [Validators.required, Validators.email], [simulateEmailCheck]],
      courseId: [null, [Validators.required, noCourseCode]],
      preferredSemester: ['Odd', Validators.required],
      agreeToTerms: [false, Validators.requiredTrue],
      additionalCourses: this.fb.array([])
    });
  }

  // Typed getter for FormArray prevents ugly casting in templates
  get additionalCourses(): FormArray {
    return this.enrollForm.get('additionalCourses') as FormArray;
  }

  addCourse(): void {
    this.additionalCourses.push(this.fb.control('', Validators.required));
  }

  removeCourse(index: number): void {
    this.additionalCourses.removeAt(index);
  }

  onSubmit(): void {
    /*
      enrollForm.value excludes controls that are disabled.
      enrollForm.getRawValue() includes all controls regardless of disabled status.
    */
    console.log('Form Value:', this.enrollForm.value);
    console.log('Raw Value:', this.enrollForm.getRawValue());
  }
}

```

```
}  
}
```

src/app/pages/reactive-enrollment-form/reactive-enrollment-form.component.html

```
<form [formGroup]="enrollForm" (ngSubmit)="onSubmit()">  
  <input formControlName="studentName" placeholder="Name" />  
  <input formControlName="studentEmail" placeholder="Email" />  
  <span *ngIf="enrollForm.get('studentEmail')?.errors?.['emailTaken']" class="error">Email taken.</span>  
  
  <input formControlName="courseId" placeholder="Course ID" />  
  <span *ngIf="enrollForm.get('courseId')?.errors?.['noCourseCode']" class="error">  
    Course code starting with XX is not allowed.  
  </span>  
  
  <div formArrayName="additionalCourses">  
    <h4>Additional Courses</h4>  
    <div *ngFor="let ctrl of additionalCourses.controls; let i=index">  
      <input [formControlName]="i" />  
      <button type="button" (click)="removeCourse(i)">Remove</button>  
    </div>  
  </div>  
  
  <button type="button" (click)="addCourse()">Add Another Course</button>  
  <button type="submit" [disabled]="enrollForm.invalid">Submit</button>  
</form>
```

HANDS-ON 6: Services & Dependency Injection

Task 1: Create and Use a Course Service

Steps 58-62: Course Interface & Singleton Service

src/app/models/course.model.ts

```
export interface Course {  
  id: number;  
  name: string;  
  code: string;  
  credits: number;  
  gradeStatus: 'passed' | 'failed' | 'pending';  
}
```

src/app/services/course.service.ts

```

import { Injectable } from '@angular/core';
import { Course } from '../models/course.model';

@Injectable({
  providedIn: 'root'
})
export class CourseService {
  private courses: Course[] = [
    { id: 1, name: 'Angular v20', code: 'CS201', credits: 4, gradeStatus: 'passed' },
    { id: 2, name: 'Data Structures', code: 'CS102', credits: 3, gradeStatus: 'passed' },
    { id: 3, name: 'Database Systems', code: 'CS204', credits: 4, gradeStatus: 'pending' },
    { id: 4, name: 'Web Engineering', code: 'CS105', credits: 3, gradeStatus: 'failed' },
    { id: 5, name: 'Cloud Architecture', code: 'CS301', credits: 3, gradeStatus: 'pending' }
  ];

  getCourses(): Course[] {
    return this.courses;
  }

  getCourseById(id: number): Course | undefined {
    return this.courses.find(c => c.id === id);
  }

  addCourse(course: Course): void {
    this.courses.push(course);
  }
}

```

Task 2: Enrollment Service and Hierarchical DI

Steps 63-67: Service-to-Service Injection & Component-level Providers

src/app/services/enrollment.service.ts

```

import { Injectable } from '@angular/core';
import { CourseService } from '../course.service';
import { Course } from '../models/course.model';

@Injectable({
  providedIn: 'root'
})
export class EnrollmentService {
  private enrolledCourseIds: number[] = [];

  constructor(private courseService: CourseService) {}

  enroll(courseId: number): void {
    if (!this.isEnrolled(courseId)) {
      this.enrolledCourseIds.push(courseId);
    }
  }

  unenroll(courseId: number): void {
    this.enrolledCourseIds = this.enrolledCourseIds.filter(id => id !== courseId);
  }

  isEnrolled(courseId: number): boolean {
    return this.enrolledCourseIds.includes(courseId);
  }

  getEnrolledCourses(): Course[] {
    return this.courseService.getCourses().filter(c => this.enrolledCourseIds.includes(c.id));
  }
}

```

src/app/components/notification/notification.component.ts

```
import { Component } from '@angular/core';
import { NotificationService } from '../../services/notification.service';

@Component({
  selector: 'app-notification',
  template: `<div class="notif">Notifications Component</div>`,
  providers: [NotificationService]
})
/*
  COMMENT (Step 67): Providing a service at component level creates a fresh instance
  scoped strictly to this component subtree rather than using the singleton root instance.
*/
export class NotificationComponent {}
```

HANDS-ON 7: Angular Routing - Guards, Lazy Loading & Route Data

Task 1: Route Configuration, Parameters and Nested Routes

Steps 68-72: Routing Table & Dynamic Navigation

src/app/app-routing.module.ts

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from '../pages/home/home.component';
import { CourseListComponent } from '../pages/course-list/course-list.component';
import { CourseDetailComponent } from '../pages/course-detail/course-detail.component';
import { StudentProfileComponent } from '../pages/student-profile/student-profile.component';
import { NotFoundComponent } from '../pages/not-found/not-found.component';
import { AuthGuard } from '../guards/auth.guard';

const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'courses',
    children: [
      { path: '', component: CourseListComponent },
      { path: ':id', component: CourseDetailComponent }
    ]
  },
  { path: 'profile', component: StudentProfileComponent, canActivate: [AuthGuard] },
  {
    path: 'enroll',
    loadChildren: () => import('../features/enrollment/enrollment.module').then(m =>
      m.EnrollmentModule)
  },
  { path: '**', component: NotFoundComponent }
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

src/app/pages/course-detail/course-detail.component.ts

```

import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { CourseService } from '../../services/course.service';
import { Course } from '../../models/course.model';

@Component({
  selector: 'app-course-detail',
  template: `
    <div *ngIf="course">
      <h2>{{ course.name }}</h2>
      <p>Code: {{ course.code }}</p>
      <p>Credits: {{ course.credits }}</p>
    </div>
  `
})
export class CourseDetailComponent implements OnInit {
  course?: Course;

  constructor(private route: ActivatedRoute, private courseService: CourseService) {}

  ngOnInit(): void {
    const id = Number(this.route.snapshot.paramMap.get('id'));
    this.course = this.courseService.getCourseById(id);
  }
}

```

Task 2: Lazy Loading and Route Guards

Steps 73-77: AuthGuard & UnsavedChangesGuard

src/app/guards/auth.guard.ts

```

import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';

@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  private isLoggedIn = true; // Hardcoded for demo

  constructor(private router: Router) {}

  canActivate(): boolean {
    if (this.isLoggedIn) return true;
    this.router.navigate(['/']);
    return false;
  }
}

```

src/app/guards/unsaved-changes.guard.ts

```

import { Injectable } from '@angular/core';
import { CanDeactivate } from '@angular/router';
import { ReactiveEnrollmentFormComponent } from '../../pages/reactive-enrollment-form/reactive-enrollment-form.component';

@Injectable({ providedIn: 'root' })
export class UnsavedChangesGuard implements CanDeactivate<ReactiveEnrollmentFormComponent> {
  canDeactivate(component: ReactiveEnrollmentFormComponent): boolean {
    if (component.enrollForm.dirty) {
      return window.confirm('You have unsaved changes. Leave?');
    }
    return true;
  }
}

```


HANDS-ON 8: HTTP Client - API Integration, Observables & Interceptors

Task 1 & Task 2: Service Refactoring & RxJS Operators

Steps 78-87: HttpClient Operations, map, catchError, retry & switchMap

src/app/services/course.service.ts (HTTP Version)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { map, catchError, tap, retry, switchMap } from 'rxjs/operators';
import { Course } from '../models/course.model';

@Injectable({ providedIn: 'root' })
export class CourseService {
  private apiUrl = 'http://localhost:3000/courses';

  constructor(private http: HttpClient) {}

  getCourses(): Observable<Course[]> {
    return this.http.get<Course[]>(this.apiUrl).pipe(
      retry(2),
      map(courses => courses.filter(c => c.credits > 0)),
      tap(courses => console.log('Courses loaded:', courses.length)),
      catchError(err => {
        console.error(err);
        return throwError(() => new Error('Failed to load courses. Please try again.'));
      })
    );
  }

  getCourseById(id: number): Observable<Course> {
    return this.http.get<Course>(`${this.apiUrl}/${id}`);
  }

  createCourse(course: Omit<Course, 'id'>): Observable<Course> {
    return this.http.post<Course>(this.apiUrl, course);
  }

  updateCourse(id: number, course: Partial<Course>): Observable<Course> {
    return this.http.put<Course>(`${this.apiUrl}/${id}`, course);
  }

  deleteCourse(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }

  /*
    switchMap cancels previous inner request if a new selection arrives before completion,
    preventing race conditions and stale response emissions.
  */
  getStudentsForCourse(courseId$: Observable<number>): Observable<any[]> {
    return courseId$.pipe(
      switchMap(id => this.http.get<any[]>(`http://localhost:3000/enrollments?courseId=${id}`))
    );
  }
}
```

Task 3: HTTP Interceptors

Steps 88-91: Auth, Error & Loading Functional Interceptors

src/app/interceptors/auth.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authReq = req.clone({
    headers: { Authorization: 'Bearer mock-token-12345' }
  });
  return next(authReq);
};
```

src/app/interceptors/error.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';

export const errorHandlerInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);
  return next(req).pipe(
    catchError(err => {
      if (err.status === 401) router.navigate(['/']);
      if (err.status === 500) console.error('Global Error: Server Error Occurred');
      return throwError(() => err);
    })
  );
};
```

HANDS-ON 9: State Management - NgRx Store, Actions, Reducers, Effects & Selectors

Task 1: Set Up NgRx Store and Define Course State

Task 2: NgRx Effects for HTTP and Enrollment State

Steps 92-100: Actions, Reducer, Selectors & Effects

src/app/store/course/course.actions.ts

```
import { createAction, props } from '@ngrx/store';
import { Course } from '../../../models/course.model';

export const loadCourses = createAction('[Course] Load Courses');
export const loadCoursesSuccess = createAction('[Course] Load Courses Success', props<{ courses: Course[] }>());
export const loadCoursesFailure = createAction('[Course] Load Courses Failure', props<{ error: string }>());
```

src/app/store/course/course.reducer.ts

```
import { createReducer, on } from '@ngrx/store';
import { Course } from '../../../models/course.model';
import * as CourseActions from './course.actions';

export interface CourseState {
  courses: Course[];
  loading: boolean;
  error: string | null;
}

export const initialState: CourseState = {
  courses: [],
  loading: false,
  error: null
};

export const courseReducer = createReducer(
  initialState,
  on(CourseActions.loadCourses, state => ({ ...state, loading: true, error: null })),
  on(CourseActions.loadCoursesSuccess, (state, { courses }) => ({ ...state, loading: false,
courses })),
  on(CourseActions.loadCoursesFailure, (state, { error }) => ({ ...state, loading: false, error }))
);
```

src/app/store/course/course.selectors.ts

```
import { createFeatureSelector, createSelector } from '@ngrx/store';
import { CourseState } from './course.reducer';

export const selectCourseState = createFeatureSelector<CourseState>('course');
export const selectAllCourses = createSelector(selectCourseState, s => s.courses);
export const selectCoursesLoading = createSelector(selectCourseState, s => s.loading);
export const selectCoursesError = createSelector(selectCourseState, s => s.error);
```

src/app/store/course/course.effects.ts

```
import { Injectable } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { CourseService } from '../../../services/course.service';
import * as CourseActions from './course.actions';
import { catchError, map, switchMap, of } from 'rxjs';

@Injectable()
export class CourseEffects {
  loadCourses$ = createEffect(() =>
    this.actions$.pipe(
      ofType(CourseActions.loadCourses),
      switchMap(() =>
        this.courseService.getCourses().pipe(
          map(courses => CourseActions.loadCoursesSuccess({ courses })),
          catchError(error => of(CourseActions.loadCoursesFailure({ error: error.message })))
        )
      )
    )
  );

  constructor(private actions$: Actions, private courseService: CourseService) {}
}
```

HANDS-ON 10: Unit Testing Angular Applications - Jasmine, Karma & TestBed

Task 1: Testing a Component - CourseCardComponent

Steps 101-105: Unit Testing Component Logic, Inputs & Outputs

`src/app/components/course-card/course-card.component.spec.ts`

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { CourseCardComponent } from './course-card.component';
import { By } from '@angular/platform-browser';
import { SimpleChange } from '@angular/core';

describe('CourseCardComponent', () => {
  let component: CourseCardComponent;
  let fixture: ComponentFixture<CourseCardComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [CourseCardComponent]
    }).compileComponents();

    fixture = TestBed.createComponent(CourseCardComponent);
    component = fixture.componentInstance;
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should display course name when @Input course is set', () => {
    component.course = { id: 1, name: 'Data Structures', code: 'CS101', credits: 4, gradeStatus:
'passed' };
    fixture.detectChanges();
    const element = fixture.debugElement.query(By.css('h3')).nativeElement;
    expect(element.textContent).toContain('Data Structures');
  });

  it('should emit enrollRequested on button click', () => {
    component.course = { id: 1, name: 'Data Structures', code: 'CS101', credits: 4, gradeStatus:
'passed' };
    fixture.detectChanges();
    spyOn(component.enrollRequested, 'emit');

    const button = fixture.debugElement.query(By.css('button')).nativeElement;
    button.click();

    expect(component.enrollRequested.emit).toHaveBeenCalled();
  });

  it('should trigger ngOnChanges on input change', () => {
    spyOn(console, 'log');
    component.ngOnChanges({
      course: new SimpleChange(null, { id: 1, name: 'Testing' }, true)
    });
    expect(console.log).toHaveBeenCalled();
  });
});
```

Task 2: Testing a Service and an NgRx-Connected Component

Steps 106-110: HttpClientTestingModule & MockStore Testing

`src/app/services/course.service.spec.ts`

```

import { TestBed } from '@angular/core';
import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';
import { CourseService } from '../course.service';

describe('CourseService', () => {
  let service: CourseService;
  let httpMock: HttpTestingController;

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule],
      providers: [CourseService]
    });
    service = TestBed.inject(CourseService);
    httpMock = TestBed.inject(HttpTestingController);
  });

  afterEach(() => {
    httpMock.verify();
  });

  it('should fetch courses via GET', () => {
    const mockCourses = [
      { id: 1, name: 'Angular', code: 'CS201', credits: 4, gradeStatus: 'passed' },
      { id: 2, name: 'React', code: 'CS202', credits: 3, gradeStatus: 'passed' }
    ];

    service.getCourses().subscribe(courses => {
      expect(courses.length).toBe(2);
    });

    const req = httpMock.expectOne('http://localhost:3000/courses');
    expect(req.request.method).toBe('GET');
    req.flush(mockCourses);
  });
});

```

src/app/pages/course-list/course-list.component.spec.ts (NgRx Connected)

```

import { ComponentFixture, TestBed } from '@angular/core';
import { CourseListComponent } from '../course-list.component';
import { provideMockStore, MockStore } from '@ngrx/store/testing';
import { By } from '@angular/platform-browser';

describe('CourseListComponent with NgRx', () => {
  let component: CourseListComponent;
  let fixture: ComponentFixture<CourseListComponent>;
  let store: MockStore;
  const initialState = { course: { courses: [], loading: true, error: null } };

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [CourseListComponent],
      providers: [provideMockStore({ initialState })]
    }).compileComponents();

    store = TestBed.inject(MockStore);
    fixture = TestBed.createComponent(CourseListComponent);
    component = fixture.componentInstance;
  });

  it('should render loading state when loading is true', () => {
    fixture.detectChanges();
    const loadingElem = fixture.debugElement.query(By.css('.loading-indicator'));
    expect(loadingElem).toBeTruthy();
  });
});

```